

Table of Contents

Table of Contents	1
Table of Figures	2
Data Type Size Exploration	3
Summary	3
Memory Segment Address Mapping	4
Summary	4
Pointer and Memory Allocation	5
Summary	5
Knowledge Check Questions	6

Table of Figures

Figure 1: Data Type Size Program	3
Figure 2: Data Type Size Output	3
Figure 3: Memory Segment Program	4
Figure 4: Memory Segment Address Output	4
Figure 5: Pointer and Heap Program	5
Figure 6: Pointer and Heap Output	5

Data Type Size Exploration

```
GNU nano 8.7.1 lab1.c
#include <stdio.h>

int main() {
printf("===Data Type Sizes on This System on This System ===\n\n");
printf("char: %zu bytes\n", sizeof(char));
printf("int: %zu bytes\n", sizeof(int));
printf("long: %zu bytes\n", sizeof(long));
printf("short: %zu bytes\n", sizeof(short));
printf("long long: %zu bytes\n", sizeof(long long));
printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
printf("float: %zu bytes\n", sizeof(float));
printf("double: %zu bytes\n", sizeof(double));
printf("pointer: %zu bytes\n", sizeof(int*));

return 0;
}
```

Figure 1: Data Type Size Program

```
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ ls
lab1.c  lab1_sizes
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ nano lab1.c
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ gcc lab1.c -o lab1_sizes
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ ./lab1_sizes
===Data Type Sizes on This System on This System ===

char: 1 bytes
int: 4 bytes
long: 8 bytes
short: 2 bytes
long long: 8 bytes
unsigned int: 4 bytes
float: 4 bytes
double: 8 bytes
pointer: 8 bytes
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$
```

Figure 2: Data Type Size Output

Summary

I wrote a C program that prints the size of each data type using `sizeof()`. After compiling and running it, the output shows char is 1 byte, int is 4, long is 8 and pointer is 8 bytes. This matches the 64-bit column of Table I.

Question: Why might `sizeof(long)` differ between 32-bit and 64-bit systems?

On a 32-bit system long is 4 bytes, and on a 64-bit system it is 8 bytes. This is because 64-bit systems use bigger addresses, so long is made bigger too.

Memory Segment Address Mapping

```
GNU nano 8.7.1 lab2_segments.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;
int bss_var;

int main() {
    int local_var = 30;
    static int local_static = 20;

    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var: %p (Data)\n", (void*)&global_var);
    printf("bss_var: %p (BSS)\n", (void*)&bss_var);
    printf("local_var: %p (Stack)\n", (void*)&local_var);
    printf("local_static: %p (Data)\n", (void*)&local_static);
    printf("heap_var ptr: %p (Static - pointer)\n", (void*)&heap_var);
    printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
```

Figure 3: Memory Segment Program

```
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ nano lab2_segments.c
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ gcc lab2_segments.c -o lab2_segments
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ ./lab2_segments
.....Variable Addresses.....

global_var: 0x5a8e75c35010 (Data)
bss_var: 0x5a8e75c3501c (BSS)
local_var: 0x7ffcaf2b06ec (Stack)
local_static: 0x5a8e75c35014 (Data)
heap_var ptr: 0x7ffcaf2b06f0 (Static - pointer)
*heap_var: 0x5a8eae0a010 (Heap - data)
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$
```

Figure 4: Memory Segment Address Output

Summary

This program prints the address of variables from each memory segment. The stack has the highest address and the Data/BSS has the lowest. The heap is in between. The gap between stack and heap is about 41 trillion bytes (0x256e007a66dc), so they are very far apart.

Pointer and Memory Allocation

```
GNU nano 8.7.1 lab3_pointer.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *forheap;

    forheap = (int * )malloc(sizeof(int));
    *forheap = 30;

    printf("Address of pointer (Stack): %p\n", (void*)&forheap);
    printf("Address of data (Heap): %p\n", (void*)forheap);
    printf("Value stores: %d\n", *forheap);

    free(forheap);
    return 0;
}
```

Figure 5: Pointer and Heap Program

```
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ nano lab3_pointer.c
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ gcc lab3_pointer.c -o lab3_pointer
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ ./lab3_pointer
Address of pointer (Stack): 0x7ffd67c2d4c0
Address of data (Heap): 0x615aba9db010
Value stores: 30
kshitiz@kshitiz-VirtualBox:~/linux_lab/lab_4$ █
```

Figure 6: Pointer and Heap Output

Summary

This program makes a pointer and gives it memory using malloc(). The pointer address is on the stack and the data address is on the heap. The value stored is 30. So the pointer lives on the stack but points to the heap.

Knowledge Check Questions

1. What is a data type in the context of programming and operating systems?

= A data type tells the size of memory to give, how to read the bytes, and what operations can be done on the data.

2. Name the five main memory segments in a process memory layout.

= The five segments are Text, Data, BSS, Heap and Stack.

3. What is the difference between the Data segment and the BSS segment?

= The Data segment keeps global and static variables that have a starting value. The BSS keeps the ones that have no starting value, and they are set to zero.

4. Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

= The stack grows downward and the heap grows upward, so they stay far apart and do not collide. The text segment is fixed, so it stays at the low addresses.

5. What happens to stack memory when a function returns? Who reclaims it?

= The stack memory of that function is removed and can be used again. It is reclaimed automatically by the compiler code and OS, and no programmer work is needed.

6. How does the OS know which memory belongs to which process?

= The OS gives each process its own page table. It maps the virtual addresses of that process to physical memory, so one process cannot use the memory of another.